



Department of Computer Science
Faculty of Computing
UNIVERSITI TEKNOLOGI MALAYSIA

SUBJECT NAME:	COMPUTER ORGANIZATION AND ARCHITECTURE				
SUBJECT CODE:	SECR 1033				
SEMESTER:	2 – 2023/2024				
LAB TITLE:	Lab 2: Arithmetic Equations & Operations				
STUDENT INFO :	Execute the lab in a group of two. <table border="1"><thead><tr><th>Student 1</th><th>Student 2</th></tr></thead><tbody><tr><td><i>No. 1, 3, 5</i></td><td><i>No 2, 4, 6</i></td></tr></tbody></table> Name 1: Iman Abadi Bin Mohd Nizwan Metric No: A23CS0084 Name 2: Muhammad naim bin Abdullah Metric No: A23CS0134	Student 1	Student 2	<i>No. 1, 3, 5</i>	<i>No 2, 4, 6</i>
Student 1	Student 2				
<i>No. 1, 3, 5</i>	<i>No 2, 4, 6</i>				
SUBMISSION DATE:	20/5/2024				

MARKS:

Arithmetic Equation Coding in Assembly Language

Q1. Execute the program below. Determine output of the program by inspecting the content of the related registers.

a) Fill in Table 1 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.

b) Paste the screenshot of all registers' content after each LINE is executed.

```
INCLUDE Irvine32.inc
.data
var1 word 1
var2 word 9

.code
main PROC
    mov ax, var1      ; LINE1
    mov bx, var2      ; LINE2
    xchg ax, bx       ; LINE3
    mov var1, ax      ; LINE4
    mov var2, bx      ; LINE5
    call DumpRegs
    exit
main ENDP
END main
```

Answer Q1

a) Fill (Write) in the contents for the related register in each line:

Table 1

LINE1	AX = 0001h var1 = 0001h	Move the value of var1 (1d) into register AX
LINE2	BX = 0009h var2 = 0009h	Move the value of var2 (9d) into register BX
LINE3	AX = 0009h BAX = 0001h	Exchange the value of register AX (1d) with register BX (9d)
LINE4	AX = 0009h var1 = 0009h	Move the value of register AX (9d) into var1
LINE5	BX = 0001h var2 = 0001h	Move the value of register AX (9d) into var1

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```
EAX = 003B0001 EBX = 0056B000 ECX = 006310AA EDX = 006310AA ESI = 006310AA  
EDI = 006310AA EIP = 00633666 ESP = 003BFEDC EBP = 003BFEE8 EFL = 00000246
```

LINE2:

```
EAX = 003B0001 EBX = 00560009 ECX = 006310AA EDX = 006310AA ESI = 006310AA  
EDI = 006310AA EIP = 00633672 ESP = 003BFEDC EBP = 003BFEE8 EFL = 00000246
```

LINE3:

```
EAX = 003B0009 EBX = 00560001 ECX = 006310AA EDX = 006310AA ESI = 006310AA  
EDI = 006310AA EIP = 00633679 ESP = 003BFEDC EBP = 003BFEE8 EFL = 00000246
```

LINE4:

```
EAX = 003B0009 EBX = 00560001 ECX = 006310AA EDX = 006310AA ESI = 006310AA  
EDI = 006310AA EIP = 00633684 ESP = 003BFEDC EBP = 003BFEE8 EFL = 00000246
```

LINE5:

```
EAX = 003B0009 EBX = 00560001 ECX = 006310AA EDX = 006310AA ESI = 006310AA  
EDI = 006310AA EIP = 00633690 ESP = 003BFEDC EBP = 003BFEE8 EFL = 00000246
```

COMMAND PROMPT:

```
EAX=00D50001  EBX=00A49000  ECX=006310AA  EDX=006310AA  
ESI=006310AA  EDI=006310AA  EBP=00D5FD98  ESP=00D5FD8C  
EIP=0063366B  EFL=00000246  CF=0   SF=0   ZF=1   OF=0   AF=0   PF=1
```

```
EAX=00D50001  EBX=00A40009  ECX=006310AA  EDX=006310AA  
ESI=006310AA  EDI=006310AA  EBP=00D5FD98  ESP=00D5FD8C  
EIP=00633677  EFL=00000246  CF=0   SF=0   ZF=1   OF=0   AF=0   PF=1
```

```
EAX=00D50009  EBX=00A40001  ECX=006310AA  EDX=006310AA  
ESI=006310AA  EDI=006310AA  EBP=00D5FD98  ESP=00D5FD8C  
EIP=0063367E  EFL=00000246  CF=0   SF=0   ZF=1   OF=0   AF=0   PF=1
```

```
EAX=00D50009  EBX=00A40001  ECX=006310AA  EDX=006310AA  
ESI=006310AA  EDI=006310AA  EBP=00D5FD98  ESP=00D5FD8C  
EIP=00633689  EFL=00000246  CF=0   SF=0   ZF=1   OF=0   AF=0   PF=1
```

```
EAX=00D50009  EBX=00A40001  ECX=006310AA  EDX=006310AA  
ESI=006310AA  EDI=006310AA  EBP=00D5FD98  ESP=00D5FD8C  
EIP=00633695  EFL=00000246  CF=0   SF=0   ZF=1   OF=0   AF=0   PF=1
```

Q2. Execute the program below. Determine output of the program by inspecting the content of the related registers and watches.

- a) Fill in Table 2 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- b) Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $Rval = (-Xval + (Yval - Zval)) + 1$

```
include irvine32.inc

.data
Rval DWORD ?
Xval DWORD 26
Yval DWORD 30
Zval DWORD 40

.code
main proc
    mov eax,Xval ; LINE1
    neg eax      ; LINE2
    mov ebx,Yval ; LINE3
    sub ebx,Zval ; LINE4
    add eax,ebx  ; LINE5
    inc eax      ; LINE6
    mov Rval,eax ; LINE7
    exit
main endp
end main
```

Answer Q2

- a) Fill (Write) in the contents for the related register in each line:

Table 2

LINE1	EAX = 0000001Ah Xval = 0000001Ah	Move the value of Xval (26d) into register EAX
LINE2	EAX = FFFFFFFE6h	reverse the sign for EAX value (1Ah) by using two's complement
LINE3	EBX = 0000001Eh Yval = 0000001Eh	Move the value of Yval (30d) into register EBX
LINE4	EBX = FFFFFFFF6h Zval = 00000028h	EBX value will be subtract by Zval (40d) and will be store in EBX
LINE5	EAX = FFFFFFFDCh EBX = FFFFFFFF6h	EAX (FFFFFFFE6h) value will be add with the value of EBX(FFFFFFF6h) and will be store in EAX
LINE6	EAX = FFFFFFFDDh	EAX value will be add by 1 and will be store in EAX
LINE7	EAX = FFFFFFFDDh Rval = FFFFFFFDDh	Move the value of EAX (FFFFFFFDDh) into the Rval

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```
Registers
EAX = 0000001A EBX = 0061E000 ECX = 00E010AA EDX = 00E010AA
ESI = 00E010AA EDI = 00E010AA EIP = 00E03665 ESP = 005EFB6C
EBP = 005EFB78 EFL = 00000246
```

LINE2:

```
Registers
EAX = FFFFFFFE6 EBX = 0061E000 ECX = 00E010AA EDX = 00E010AA
ESI = 00E010AA EDI = 00E010AA EIP = 00E0366C ESP = 005EFB6C
EBP = 005EFB78 EFL = 00000293
```

LINE3:

```
Registers
EAX = FFFFFFFE6 EBX = 0000001E ECX = 00E010AA EDX = 00E010AA
ESI = 00E010AA EDI = 00E010AA EIP = 00E03677 ESP = 005EFB6C
EBP = 005EFB78 EFL = 00000293
```

LINE4:

```
Registers
EAX = FFFFFFFE6 EBX = FFFFFFFF6 ECX = 00E010AA EDX = 00E010AA
ESI = 00E010AA EDI = 00E010AA EIP = 00E03682 ESP = 005EFB6C
EBP = 005EFB78 EFL = 00000287
```

LINE5:

```
Registers
EAX = FFFFFFFDC EBX = FFFFFFFF6 ECX = 00E010AA EDX = 00E010AA
ESI = 00E010AA EDI = 00E010AA EIP = 00E03689 ESP = 005EFB6C
EBP = 005EFB78 EFL = 00000283
```

LINE6:

```
Registers
EAX = FFFFFFFDD EBX = FFFFFFFF6 ECX = 00E010AA EDX = 00E010AA
ESI = 00E010AA EDI = 00E010AA EIP = 00E0368F ESP = 005EFB6C
EBP = 005EFB78 EFL = 00000287
```

LINE7:

```
Registers
EAX = FFFFFFFDD EBX = FFFFFFFF6 ECX = 00E010AA EDX = 00E010AA
ESI = 00E010AA EDI = 00E010AA EIP = 00E03699 ESP = 00F0F8F4
EBP = 00F0F900 EFL = 00000287
```

Watch 1	
Search (Ctrl+E)    Search Depth: 3	
Name	Value
 Rval	0xffffffff
 Xval	0x0000001a
 Yval	0x0000001e
 Zval	0x00000028

COMMAND PROMPT :

```
EAX=0000001A  EBX=0101E000  ECX=00E010AA  EDX=00E010AA
ESI=00E010AA  EDI=00E010AA  EBP=00F0F900  ESP=00F0F8F4
EIP=00E0366A  EFL=00000246  CF=0  SF=0  ZF=1  OF=0  AF=0  PF=1
```

```
EAX=FFFFFFE6  EBX=0101E000  ECX=00E010AA  EDX=00E010AA
ESI=00E010AA  EDI=00E010AA  EBP=00F0F900  ESP=00F0F8F4
EIP=00E03671  EFL=00000293  CF=1  SF=1  ZF=0  OF=0  AF=1  PF=0
```

```
EAX=FFFFFFE6  EBX=0000001E  ECX=00E010AA  EDX=00E010AA
ESI=00E010AA  EDI=00E010AA  EBP=00F0F900  ESP=00F0F8F4
EIP=00E0367C  EFL=00000293  CF=1  SF=1  ZF=0  OF=0  AF=1  PF=0
```

```
EAX=FFFFFFE6  EBX=FFFFFFFF  ECX=00E010AA  EDX=00E010AA
ESI=00E010AA  EDI=00E010AA  EBP=00F0F900  ESP=00F0F8F4
EIP=00E03687  EFL=00000287  CF=1  SF=1  ZF=0  OF=0  AF=0  PF=1
```

```
EAX=FFFFFFDC  EBX=FFFFFFFF  ECX=00E010AA  EDX=00E010AA
ESI=00E010AA  EDI=00E010AA  EBP=00F0F900  ESP=00F0F8F4
EIP=00E0368E  EFL=00000283  CF=1  SF=1  ZF=0  OF=0  AF=0  PF=0
```

```
EAX=FFFFFFDD  EBX=FFFFFFFF  ECX=00E010AA  EDX=00E010AA
ESI=00E010AA  EDI=00E010AA  EBP=00F0F900  ESP=00F0F8F4
EIP=00E03694  EFL=00000287  CF=1  SF=1  ZF=0  OF=0  AF=0  PF=1
```

```
EAX=FFFFFFDD  EBX=FFFFFFFF  ECX=00E010AA  EDX=00E010AA
ESI=00E010AA  EDI=00E010AA  EBP=00F0F900  ESP=00F0F8F4
EIP=00E0369E  EFL=00000287  CF=1  SF=1  ZF=0  OF=0  AF=0  PF=1
```

Q3. Execute the program below. Determine output of the program by inspecting the content of the related registers.

- Fill in Table 3 with the content of each register or variable on every LINE, in **Hexadecimal** (as per the output). Please complete the comments for every LINE.
- Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = [(\text{var1} * \text{var2}) + \text{var3}] - 1$

```
include irvine32.inc

.data
var1 DWORD 5
var2 DWORD 10
var3 DWORD 20
var4 DWORD ?

.code
main proc
    mov eax, var1          ; LINE1
    mul var2               ; LINE2
    add eax, var3          ; LINE3
    dec eax               ; LINE4
    mov var4, eax          ; LINE5
    exit
main endp
end main
```

Answer Q3

- Fill (Write) in the contents for the related register in each line:

Table 3

LINE1	EAX = 00000005h var1 = 00000005h	Move the value of var1 (5d) into register EAX
LINE2	EAX = 00000032h var2 = 0000000Ah	Multiply register EAX (5d) with the value of var2 (10d) and the result is stored in EAX (50d)
LINE3	EAX = 00000046h var3 = 00000014h	Adds the the value of register EAX (50d) with the value of var3 (20d)
LINE4	EAX = 00000045h var4 = 00000000h	Deduct 1 from register EAX (70d) to become (69d)
LINE 5	EAX = 00000045h var4 = 00000045h	Move the value of register EAX into var4 (69d)

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

```
EAX = 00000005 EBX = 002B9000 ECX = 00DE10AA EDX = 00DE10AA ESI = 00DE10AA
EDI = 00DE10AA EIP = 00DE3665 ESP = 004FFA60 EBP = 004FFA6C EFL = 00000246
```

LINE2:

```
EAX = 00000032 EBX = 002B9000 ECX = 00DE10AA EDX = 00000000 ESI = 00DE10AA
EDI = 00DE10AA EIP = 00DE3670 ESP = 004FFA60 EBP = 004FFA6C EFL = 00000202
```

LINE3:

```
EAX = 00000046 EBX = 002B9000 ECX = 00DE10AA EDX = 00000000 ESI = 00DE10AA
EDI = 00DE10AA EIP = 00DE367B ESP = 004FFA60 EBP = 004FFA6C EFL = 00000202
```

LINE4:

```
EAX = 00000045 EBX = 002B9000 ECX = 00DE10AA EDX = 00000000 ESI = 00DE10AA
EDI = 00DE10AA EIP = 00DE3681 ESP = 004FFA60 EBP = 004FFA6C EFL = 00000202
```

COMMAND PROMPT:

```
EAX=00000005 EBX=005D4000 ECX=00DE10AA EDX=00DE10AA
ESI=00DE10AA EDI=00DE10AA EBP=0073FAB0 ESP=0073FAA4
EIP=00DE366A EFL=00000246 CF=0 SF=0 ZF=1 OF=0 AF=0 PF=1
```

```
EAX=00000032 EBX=005D4000 ECX=00DE10AA EDX=00000000
ESI=00DE10AA EDI=00DE10AA EBP=0073FAB0 ESP=0073FAA4
EIP=00DE3675 EFL=00000202 CF=0 SF=0 ZF=0 OF=0 AF=0 PF=0
```

```
EAX=00000046 EBX=005D4000 ECX=00DE10AA EDX=00000000
ESI=00DE10AA EDI=00DE10AA EBP=0073FAB0 ESP=0073FAA4
EIP=00DE3680 EFL=00000202 CF=0 SF=0 ZF=0 OF=0 AF=0 PF=0
```

```
EAX=00000045 EBX=005D4000 ECX=00DE10AA EDX=00000000
ESI=00DE10AA EDI=00DE10AA EBP=0073FAB0 ESP=0073FAA4
EIP=00DE3686 EFL=00000202 CF=0 SF=0 ZF=0 OF=0 AF=0 PF=0
```

Q4. Execute the program below. Determine output of the program by inspecting the content of the related registers.

- Fill in Table 4 with the content of each register or variable on every LINE, in Hexadecimal (as per the output). Please complete the comments for every LINE.
- Paste the screenshot of all registers' content after each LINE is executed.

Arithmetic expression: $\text{var4} = (\text{var1} * 5) / (\text{var2} - 3)$

```
include Irvine32.inc
.data
    var1 WORD 40
    var2 WORD 10
    var4 WORD ?
.code
main proc
    mov ax,var1           ; LINE1
    mov bx,5              ; LINE2
    mul bx                ; LINE3
    mov bx,var2           ; LINE4
    sub bx,3              ; LINE5
    div bx                ; LINE6
    mov var4,ax           ; LINE7
    exit
main endp
end main
```

Answer Q4

- Fill (Write) in the contents for the related register in each line:

Table 4

LINE1	AX = 0028h var1 = 0028h	Move the value of var1 (40d) into register AX
LINE2	BX = 0005h	Move the value of 5h into register BX
LINE3	AX = 00C8h BX = 0005h	AX (28h) will be multiply by BX (5h) and store it in AX
LINE4	BX = 000Ah var2 = 000Ah	Move the value of var2 (Ah) into register BX
LINE5	BX = 0007h	BX value (Ah) will be subtract with the value of 3d and store in BX
LINE6	AX = 001C BX = 0007 DX = 0004	AX (C8h) is dividend and BX is divisor (7h) . the Quotient of operation between AX and BX will be store in AX while the reminder will be store in DX
LINE7	AX = 001C var4 = 001C	Move the value of AX (1Ch) into var4

b) Paste here screenshot of all registers' content after each LINE is executed:

LINE1:

Registers

EAX = 008F0028 EBX = 00656000 ECX = 00F010AA EDX = 00F010AA
ESI = 00F010AA EDI = 00F010AA EIP = 00F03666 ESP = 008FF93C
EBP = 008FF948 EFL = 00000246

LINE2:

Registers

EAX = 008F0028 EBX = 00650005 ECX = 00F010AA EDX = 00F010AA
ESI = 00F010AA EDI = 00F010AA EIP = 00F0366F ESP = 008FF93C
EBP = 008FF948 EFL = 00000246

LINE3:

Registers

EAX = 008F00C8 EBX = 00650005 ECX = 00F010AA EDX = 00F00000
ESI = 00F010AA EDI = 00F010AA EIP = 00F03677 ESP = 008FF93C
EBP = 008FF948 EFL = 00000202

LINE4:

Registers

EAX = 008F00C8 EBX = 0065000A ECX = 00F010AA EDX = 00F00000
ESI = 00F010AA EDI = 00F010AA EIP = 00F03683 ESP = 008FF93C
EBP = 008FF948 EFL = 00000202

LINE5:

Registers

EAX = 008F00C8 EBX = 00650007 ECX = 00F010AA EDX = 00F00000
ESI = 00F010AA EDI = 00F010AA EIP = 00F0368C ESP = 008FF93C
EBP = 008FF948 EFL = 00000202

LINE6:

Registers

EAX = 008F001C EBX = 00650007 ECX = 00F010AA EDX = 00F00004
ESI = 00F010AA EDI = 00F010AA EIP = 00F03694 ESP = 008FF93C
EBP = 008FF948 EFL = 00000202

LINE7:

Watch 1	
Search (Ctrl+E)    Search Depth: 3 	
Name	Value
 var1	40
 var2	10
 var4	28
...	

Registers

EAX = 008F001C EBX = 00650007 ECX = 00F010AA EDX = 00F00004
ESI = 00F010AA EDI = 00F010AA EIP = 00F0369F ESP = 008FF93C
EBP = 008FF948 EFL = 00000202

COMMAND PROMPT:

```
EAX=008F0028 EBX=00656000 ECX=00F010AA EDX=00F010AA  
ESI=00F010AA EDI=00F010AA EBP=008FF948 ESP=008FF93C  
EIP=00F0366B EFL=00000246 CF=0 SF=0 ZF=1 OF=0 AF=0 PF=1
```

```
EAX=008F0028 EBX=00650005 ECX=00F010AA EDX=00F010AA  
ESI=00F010AA EDI=00F010AA EBP=008FF948 ESP=008FF93C  
EIP=00F03674 EFL=00000246 CF=0 SF=0 ZF=1 OF=0 AF=0 PF=1
```

```
EAX=008F00C8 EBX=00650005 ECX=00F010AA EDX=00F00000  
ESI=00F010AA EDI=00F010AA EBP=008FF948 ESP=008FF93C  
EIP=00F0367C EFL=00000202 CF=0 SF=0 ZF=0 OF=0 AF=0 PF=0
```

```
EAX=008F00C8 EBX=0065000A ECX=00F010AA EDX=00F00000  
ESI=00F010AA EDI=00F010AA EBP=008FF948 ESP=008FF93C  
EIP=00F03688 EFL=00000202 CF=0 SF=0 ZF=0 OF=0 AF=0 PF=0
```

```
EAX=008F00C8 EBX=00650007 ECX=00F010AA EDX=00F00000  
ESI=00F010AA EDI=00F010AA EBP=008FF948 ESP=008FF93C  
EIP=00F03691 EFL=00000202 CF=0 SF=0 ZF=0 OF=0 AF=0 PF=0
```

```
EAX=008F001C EBX=00650007 ECX=00F010AA EDX=00F00004  
ESI=00F010AA EDI=00F010AA EBP=008FF948 ESP=008FF93C  
EIP=00F03699 EFL=00000202 CF=0 SF=0 ZF=0 OF=0 AF=0 PF=0
```

```
EAX=008F001C EBX=00650007 ECX=00F010AA EDX=00F00004  
ESI=00F010AA EDI=00F010AA EBP=008FF948 ESP=008FF93C  
EIP=00F036A4 EFL=00000202 CF=0 SF=0 ZF=0 OF=0 AF=0 PF=0
```

Short Notes for MUL CX and DIV BL:

MUL CX

- a. MUL always uses AX (or its extended versions EAX or RAX) as the implicit destination register.
- b. The operand size determines the size of the result:
 - i. Byte-sized operand: Result in AX
 - ii. Word-sized operand: Result in DX:AX
 - iii. Doubleword-sized operand (32-bit mode): Result in EDX:EAX
 - iv. Quadword-sized operand (64-bit mode): Result in RDX:RAX
- c. The upper half of the result (DX or EDX or RDX) holds any overflow bits.
- d. The Carry Flag (CF) is set if the upper half of the product is non-zero.

DIV BL

- a. DIV always uses the DX:AX or EDX:EAX pair as the implicit dividend register.
 - b. The divisor is specified as the operand of the DIV instruction.
 - c. The quotient is stored in AX (for 16-bit division) or EAX (for 32-bit division).
 - d. The remainder is stored in DX.
 - e. Clear DX (or EDX for 32-bit division) before division to ensure a correct 16-bit or 32-bit dividend.
 - f. If the divisor is 0, a division error occurs.
 - g. The Overflow Flag (OF) is set if the quotient is too large to fit in the destination register.
-

Q5. Given the following instructions as is Code Snippet 1.

- a) Write a full program to execute the Code Snippet 1.
- b) What are the contents of the related registers after Code Snippet 1 is executed? Paste the screenshot of DumpReg.

; Code Snippet 1 (MUL CX)

```
MOV DX,0      ; Clear DX
MOV AX,1000h   ; Load 1000h into AX
MOV CX, 25h    ; Load 25h into CX
MUL CX        ; Multiply AX by CX, storing the result in DX:AX
```

Answer Q5

- a) Screenshot of full program (.asm) :

```
1  include irvine32.inc
2  .data
3  .code
4  main proc
5      mov dx,0      ; Clear DX
6      call DumpRegs ≤ 1ms elapsed
7      mov ax,1000h   ; Load 1000h into AX
8      call DumpRegs
9      mov cx, 25h    ; Load 25h into CX
10     call DumpRegs
11     mul cx         ; Multiply AX by CX, storing the result in DX:AX
12     call DumpRegs
13     exit
14 main endp
15 end main
```

- b) Paste here the screenshot of the final registers' content (DumpReg):

```
EAX=00935000  EBX=006D9000  ECX=000F0025  EDX=000F0002
ESI=000F10AA  EDI=000F10AA  EBP=0093FB5C  ESP=0093FB50
EIP=000F3674  EFL=00000A07  CF=1  SF=0  ZF=0  OF=1  AF=0  PF=1
```

Q6. Given the following instructions as is Code Snippet 2.

- a) Write a full program to execute the Code Snippet 2.
- b) What are the contents of the related registers after Code Snippet 2 is executed? Paste the screenshot of DumpReg.

; Code Snippet 2 (DIV BL)

```
MOV DX, 0      ; Clear DX to form the 16-bit dividend in DX:AX
MOV AX, 803h   ; Load the dividend (8003h) into AX
MOV BL, 10h    ; Load the divisor (10h) into BL
DIV BL         ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
```

Answer Q6

- a) Screenshot of full program (.asm) :

```
1  include Irvine32.inc
2  .data
3  .code
4  main proc
5  MOV DX, 0      ; Clear DX to form the 16-bit dividend in DX:AX
6  MOV AX, 803h   ; Load the dividend (8003h) into AX
7  MOV BX, 10h    ; Load the divisor (10h) into BL
8  DIV BX         ; Divide DX:AX by BL, whereby AX=quotient & DX=remainder
9  call DumpRegs
10 exit
11 main ENDP
12 END main
```

- b) Paste here the screenshot of the final registers' content (DumpReg):

```
EAX=006F0380  EBX=00418010  ECX=008410AA  EDX=00840000
ESI=008410AA  EDI=008410AA  EBP=006FFA08  ESP=006FF9FC
EIP=00843671  EFL=00000246  CF=0  SF=0  ZF=1  OF=0  AF=0  PF=1
```

Youtube Link : <https://youtu.be/L1eNULlqO3o>